Penetration Testing Report

Contributors Name: Egeonu Chioma Lilian
Program: HCPT
Date: 28.02.25

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 3 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 3 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	**Cross-Site Request Forgery, Cross-Origin Resource Sharing**

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 3 Labs**.

Total number of Sub-labs: {count} Sub-labs

High	Medium	Low
5	4	5

High - **Number of Sub-labs with hard difficulty level**

Medium - **Number of Sub-labs with Medium difficulty level**

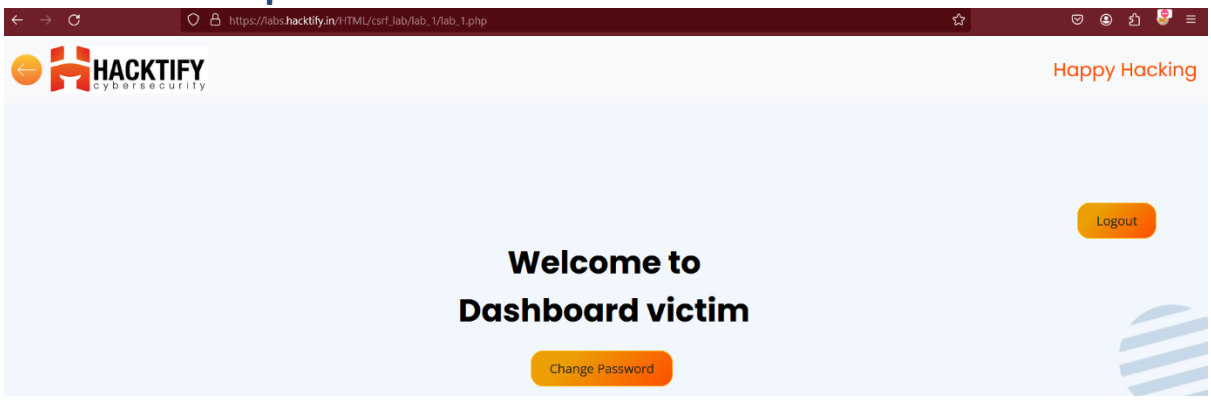
Low - **Number of Sub-labs with Easy difficulty level**

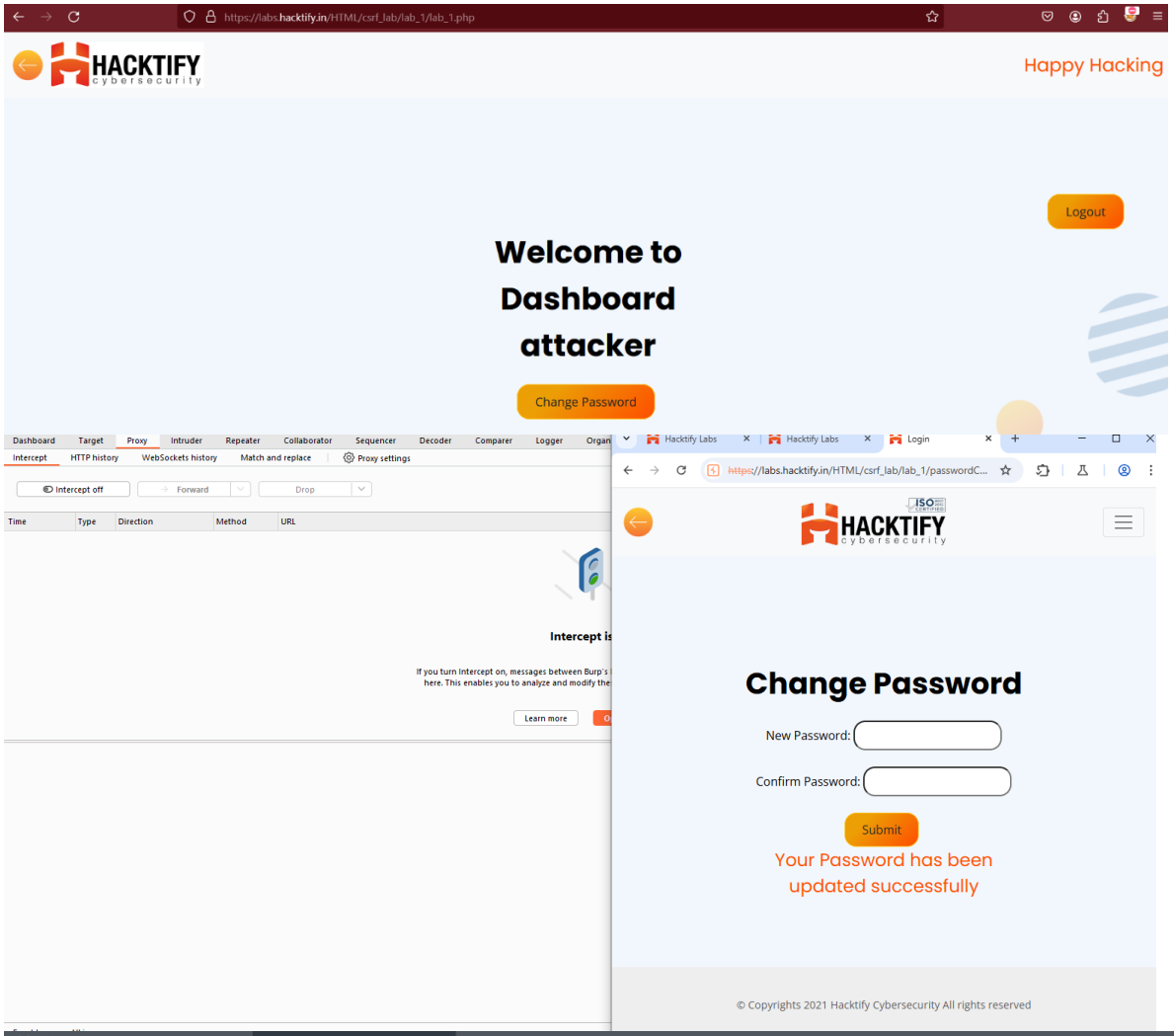
1. Cross-Site Request Forgery

1.1. Eassyy CSRF

Reference	Risk Rating
Eassyy CSRF	**Low**
Tools Used	
Burp Suite and Hacktify Poc Generator.	
Vulnerability Description	
In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.	
How It Was Discovered	
Manual Testing and analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_1/index.php	
Consequences of not Fixing the Issue	
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions. Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.	
Suggested Countermeasures	
Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.	
References	
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html	

Proof of Concept





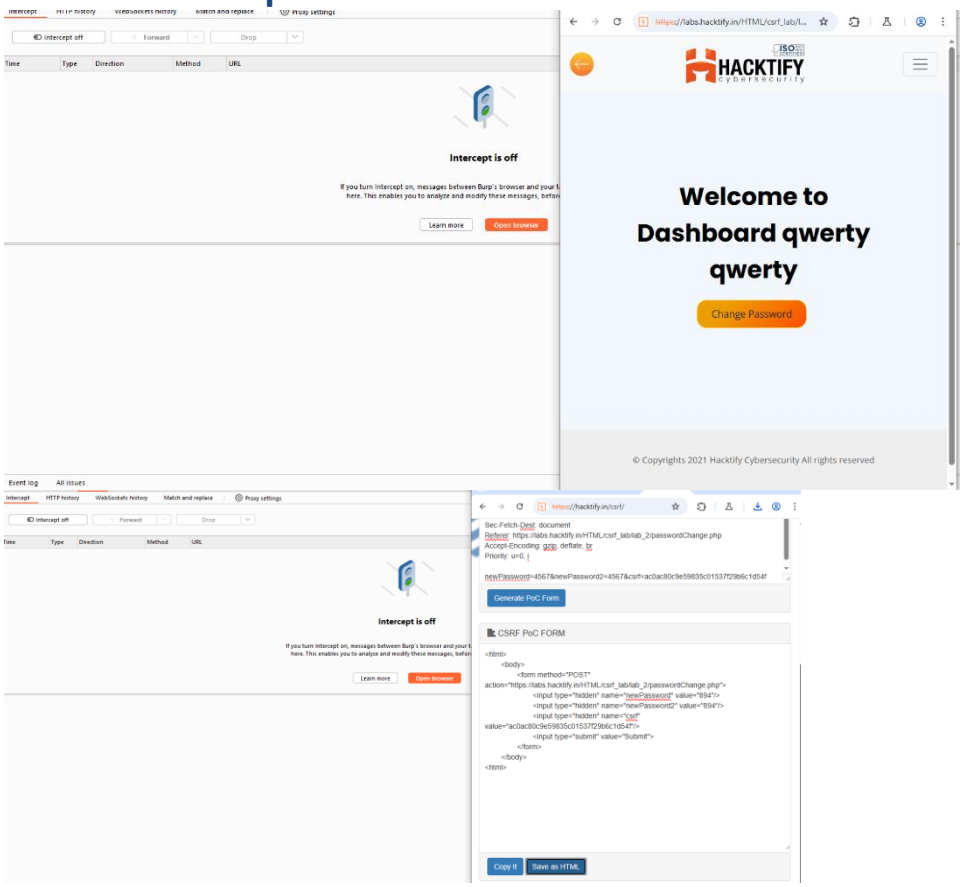
flag | csrf-poc.html

```html
<html>
    <body>
        <form method="POST" action="https://labs.hacktify.in/HTML/csrf_lab/lab_1/passwordChange.php">
            <input type="hidden" name="newPassword" value="1234"/>
            <input type="hidden" name="newPassword2" value="1234"/>
            <input type="submit" value="Submit">
        </form>
    </body>
<html>
```

1.2. Always Validate Tokens

Reference	Risk Rating
Always Validate Tokens	**Medium**
Tools Used	
Burp Suite and Hacktify Poc Generator.	
Vulnerability Description	
In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.	
How It Was Discovered	
Manual Testing and analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_2/index.php	
Consequences of not Fixing the Issue	
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions. Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.	
Suggested Countermeasures	
Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.	
References	
https://owasp.org/www-project-web-security-testing-guide/latest/	

Proof of Concept



1.3. I Hate When Someone Uses My Tokens!

Reference	Risk Rating
I Hate When Someone Uses My Tokens!	**Medium**
Tools Used	
Burp Suite and Hacktify Poc Generator.	
Vulnerability Description	
In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.	
How It Was Discovered	
Manual Testing and analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_4/index.php	
Consequences of not Fixing the Issue	
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions. Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.	
Suggested Countermeasures	

Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.
References
https://owasp.org/www-project-web-security-testing-guide/latest/

Proof of Concept

```
POST /HTML/csrf_lab/lab_4/passwordChange.php HTTP/2
Host: labs.hacktify.in
Cookie: sbjs_migrations=1418474375998%3D1; sbjs_current_add=
fd%3D2024-02-24%2011%3A00%3A38%7C%7C%7Cep%3Dhttps%3A%2F%2Fhacktify.in%2F%7C%7C%7Crf%3Dhttps%3A%2F%2Fw
ww.google.com%2F; sbjs_first_add=
fd%3D2024-02-24%2010%3A45%3A25%7C%7C%7Cep%3Dhttps%3A%2F%2Fhacktify.in%2F%7C%7C%7Crf%3Dhttps%3A%2F%2Fw
ww.google.com%2F; sbjs_current=
typ%3Dorganic%7C%7C%7Csrc%3Dgoogle%7C%7C%7Cmdm%3Dorganic%7C%7C%7Ccmp%3D%28none%29%7C%7C%7Ccnt%3D%28no
ne%29%7C%7C%7Ctrm%3D%28none%29%7C%7C%7Cid%3D%28none%29; sbjs_first=
typ%3Dorganic%7C%7C%7Csrc%3Dgoogle%7C%7C%7Cmdm%3Dorganic%7C%7C%7Ccmp%3D%28none%29%7C%7C%7Ccnt%3D%28no
ne%29%7C%7C%7Ctrm%3D%28none%29%7C%7C%7Cid%3D%28none%29; sbjs_udata=
vst%3D1%7C%7C%7Cuip%3D%28none%29%7C%7C%7Cuag%3DMozilla%2F5.0%20%28Windows%20NT%2010.0%3B%20Win64%3B%2
0x64%3B%20rv%3A123.0%29%20Gecko%2F20100101%20Firefox%2F123.0; PHPSESSID=
560e0e3ff14fa94bf300376ae9c7005d
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 76
Origin: https://labs.hacktify.in
Referer: https://labs.hacktify.in/HTML/csrf_lab/lab_4/passwordChange.php
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Te: trailers

newPassword=123456&newPassword2=123456&csrf=34b577be20fbc15477aadb9a08101ff9
```

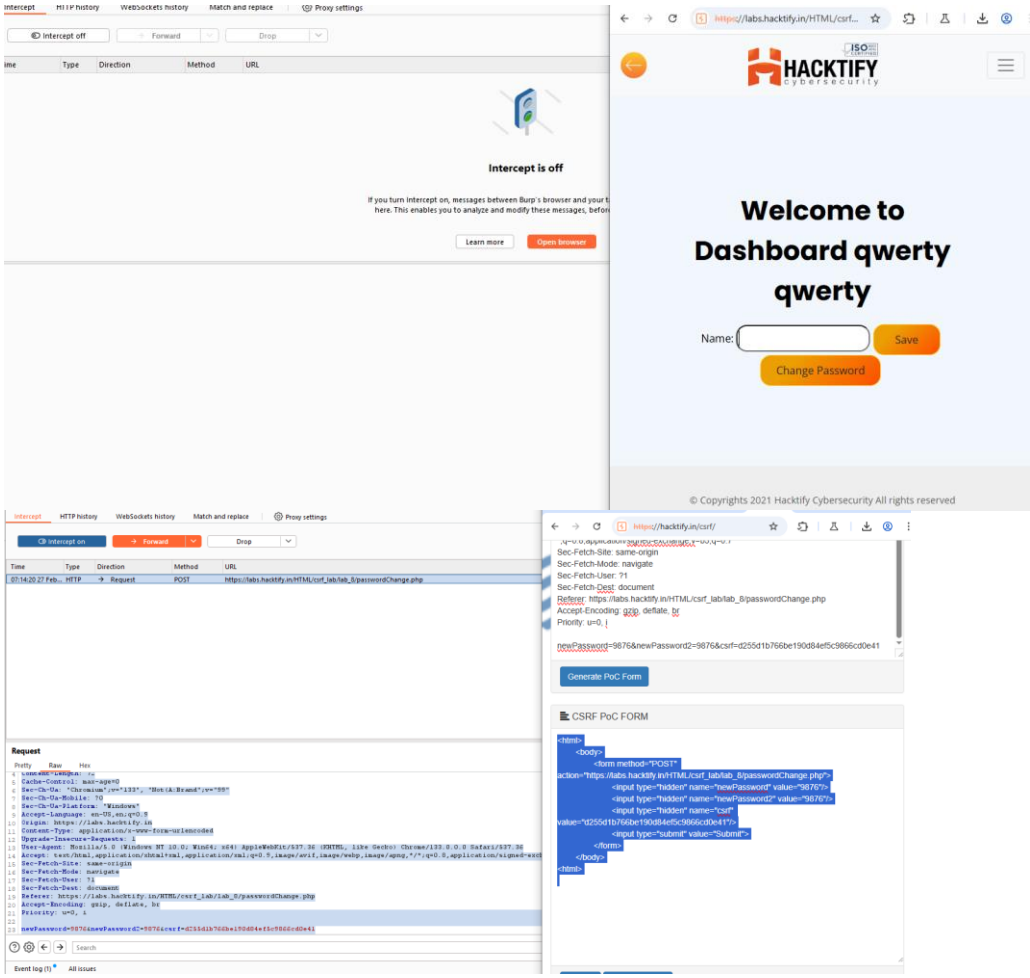
```html
<html>
    <body>
        <form method="POST" action="https://labs.hacktify.in/HTML/csrf_lab/lab_4/passwordChange.php">
            <input type="hidden" name="newPassword" value="123456"/>
            <input type="hidden" name="newPassword2" value="123456"/>
            <input type="hidden" name="csrf" value="34b577be20fbc15477aadb9a08101ff9"/>
            <input type="submit" value="Submit">
        </form>
    </body>
<html>
```

1.4. GET Me or POST Me

Reference	Risk Rating
GET Me or POST Me	**Easy**

Tools Used
Burp Suite and Hacktify Poc Generator.
Vulnerability Description
In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.
How It Was Discovered
Manual Testing and analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/csrf_lab/lab_6/index.php
Consequences of not Fixing the Issue
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions. Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.
Suggested Countermeasures
Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.
References
https://portswigger.net/web-security/csrf/cheat-sheet

Proof of Concept

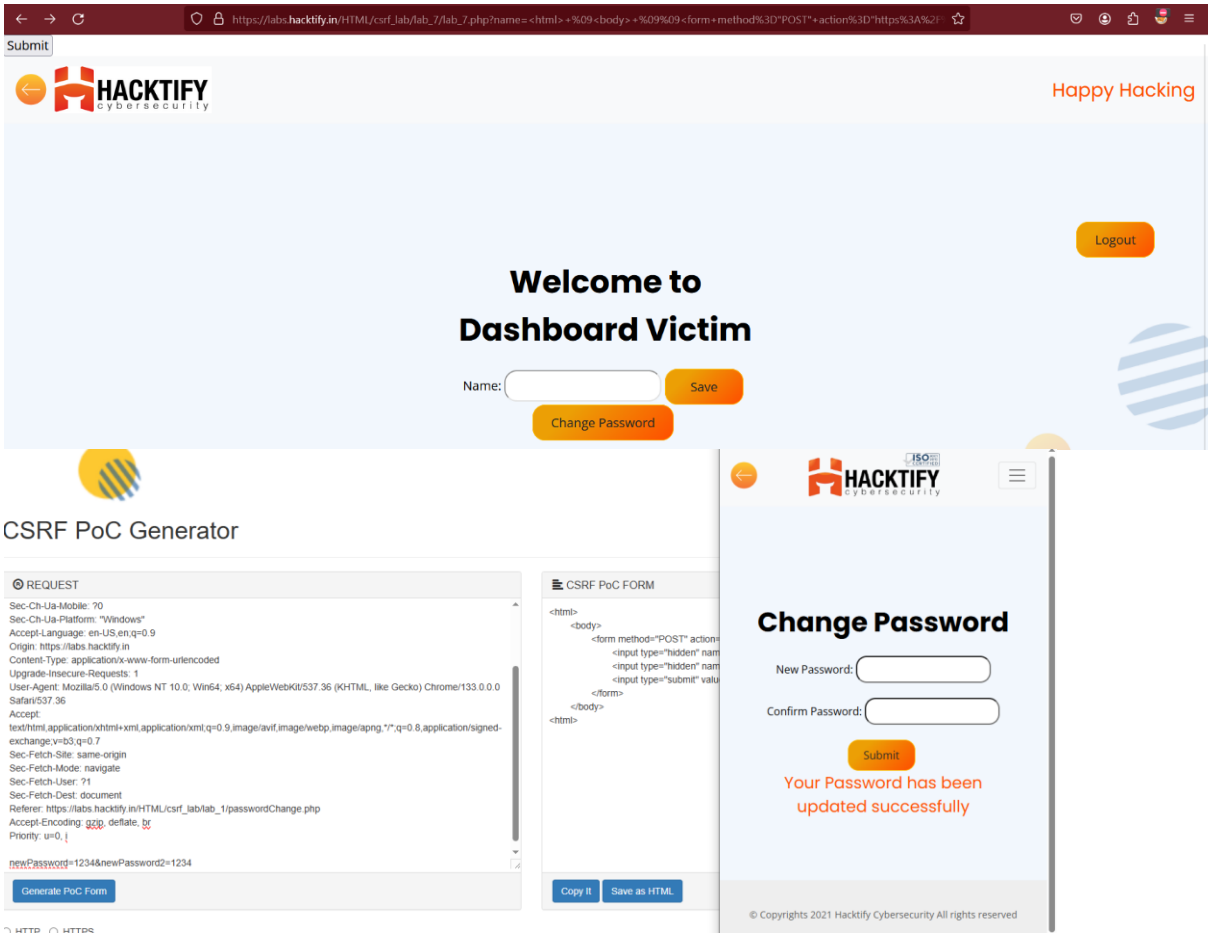


1.5. XSS The Saviour

Reference	Risk Rating
XSS The Saviour	**Hard**
Tools Used	
Burp Suite and Hacktify Poc Generator.	
Vulnerability Description	
In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.	
How It Was Discovered	
Manual Testing and analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_7/index.php	
Consequences of not Fixing the Issue	
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions.	

Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.
Suggested Countermeasures
Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.
References
https://owasp.org/www-project-web-security-testing-guide/latest/

Proof of Concept

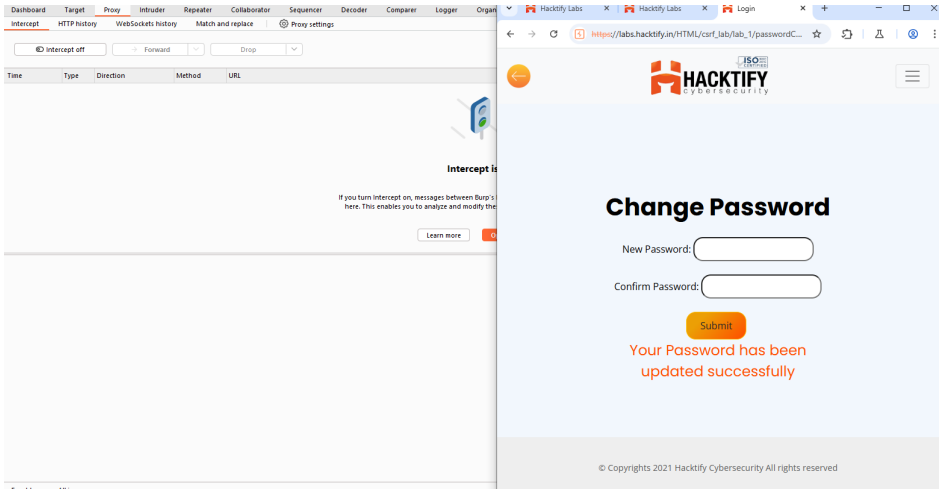


1.6. Rm -Rf Token

Reference	Risk Rating
Rm -Rf Token	**Hard**
Tools Used	
Burp Suite and Hacktify Poc Generator.	
Vulnerability Description	

In the user account management functionality of a website, a CSRF (Cross-Site Request Forgery) vulnerability exists, allowing attackers to perform unauthorized actions on behalf of authenticated users.
How It Was Discovered
Manual Testing and analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/csrf_lab/lab_8/index.php
Consequences of not Fixing the Issue
Unauthorized actions performed on behalf of authenticated users, such as changing passwords, modifying account settings, or initiating financial transactions. Compromise of user accounts, leading to potential data loss, identity theft, or unauthorized access to sensitive information.
Suggested Countermeasures
Include anti-CSRF tokens or synchronizer tokens in forms and requests to validate the origin of requests and prevent CSRF attacks. Implement strict validation of requests to ensure that sensitive actions, such as password changes, require explicit user consent and authentication.
References
https://owasp.org/www-project-web-security-testing-guide/latest/

Proof of Concept



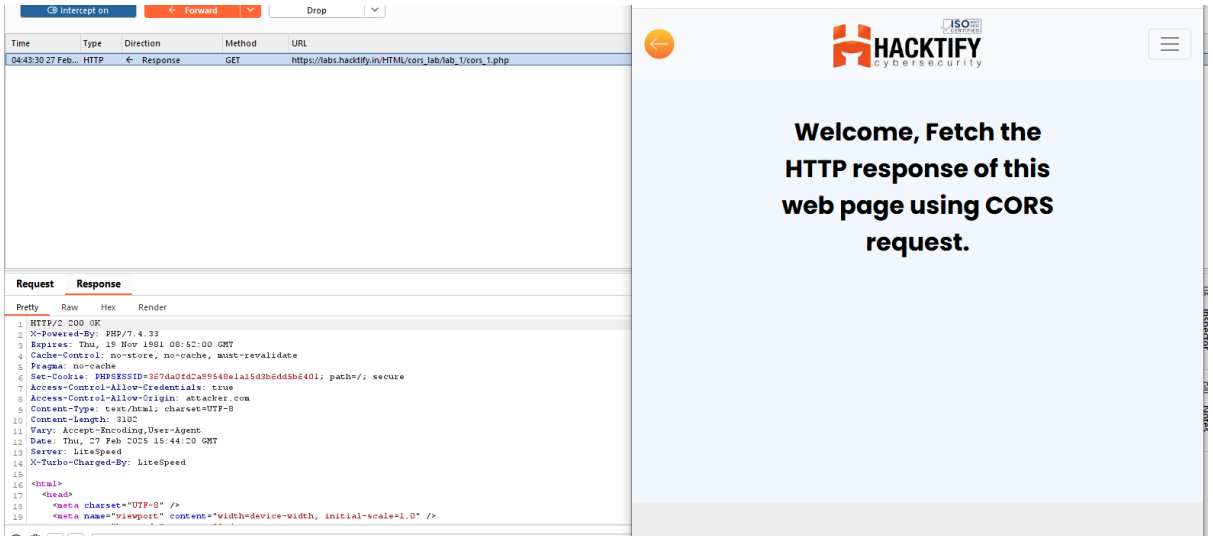
2. Cross-Origin Resource Sharing

2.1. Cors With Arbitrary Origin

Reference	Risk Rating
Cors With Arbitrary Origin	**Easy**
Tools Used	
Manual analysis and Burp-Suite	
Vulnerability Description	
A CORS vulnerability with an arbitrary origin is when a web server is configured to allow requests from any website, essentially disabling the Same-Origin Policy, which could enable malicious actors to access sensitive data from the server by making requests from their own domain, regardless of whether they are authorized to do so.	

How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/cors_lab/lab_1/index.php
Consequences of not Fixing the Issue
Data theft, Cross-Site Scripting (XSS) amplification, A malicious website could use a CORS vulnerability to enhance the impact of an XSS attack by making requests to the vulnerable server to retrieve additional data.
Suggested Countermeasures
Whitelisting of origins such as to, always explicitly define which domains are allowed to access your server's resources by setting the Access-Control-Allow-Origin header to a specific list of trusted origins. Validate requests; Implement additional checks on the server-side to verify the legitimacy of requests even if they come from a seemingly allowed origin.
References
https://portswigger.net/kb/issues/00200601_cross-origin-resource-sharing-arbitrary-origin-trusted

Proof of Concept

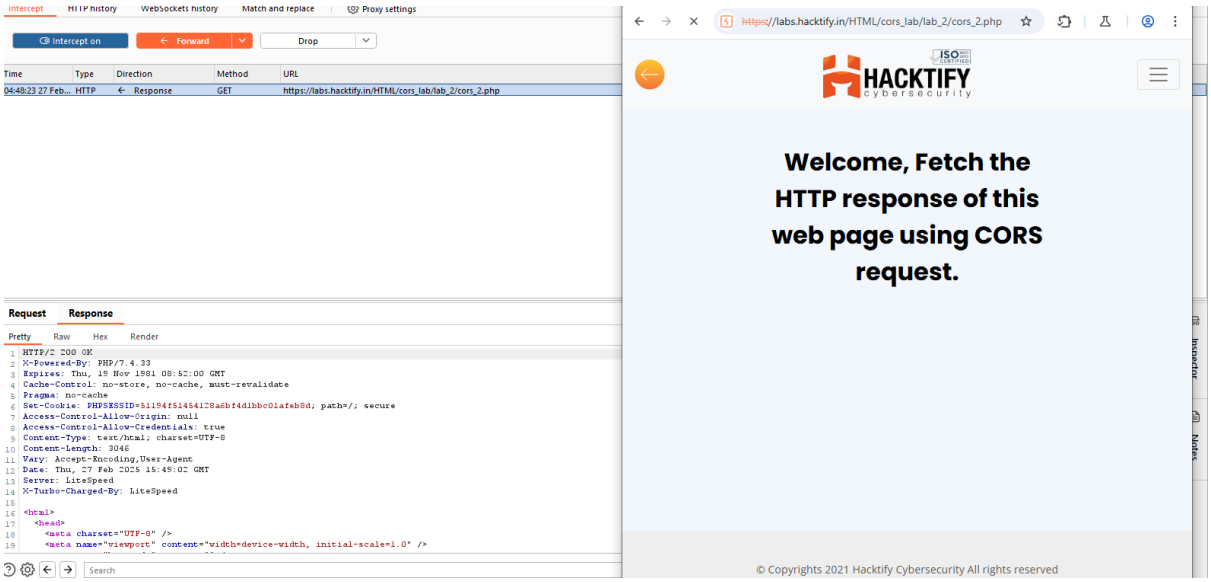


2.2. Cors with Null origin

Reference	Risk Rating
Cors with Null origin	**Low**
Tools Used	
Manual analysis and Burp-Suite	
Vulnerability Description	
This vulnerability relates to CORS origin validation accepting a null origin. When a Gradio server is deployed locally, the localhost_aliases variable includes "null" as a valid origin. This allows attackers to make unauthorized requests from sandboxed iframes or other sources with a null origin, potentially leading to data theft, such as user authentication tokens or uploaded files. This impacts users running Gradio locally, especially those using basic authentication.	
How It Was Discovered	

Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/cors_lab/lab_2/index.php
Consequences of not Fixing the Issue
There is a drastic impact on the security of web applications when there is a CORS vulnerability. An attacker can bypass the same-origin policy of a web browser to perform unauthorized transactions or gain access to sensitive data..
Suggested Countermeasures
Proper configuration of cross-origin requests If a web resource contains sensitive information, the origin should be properly specified in the Access-Control-Allow-Origin header, only allow trusted sites, Avoid wildcards in internal networks Avoid using wildcards in internal networks. Trusting network configuration alone to protect internal resources is not sufficient when internal browsers can access untrusted external domains.
References
https://portswigger.net/web-security/cors#what-is-cors-cross-origin-resource-sharing

Proof of Concept

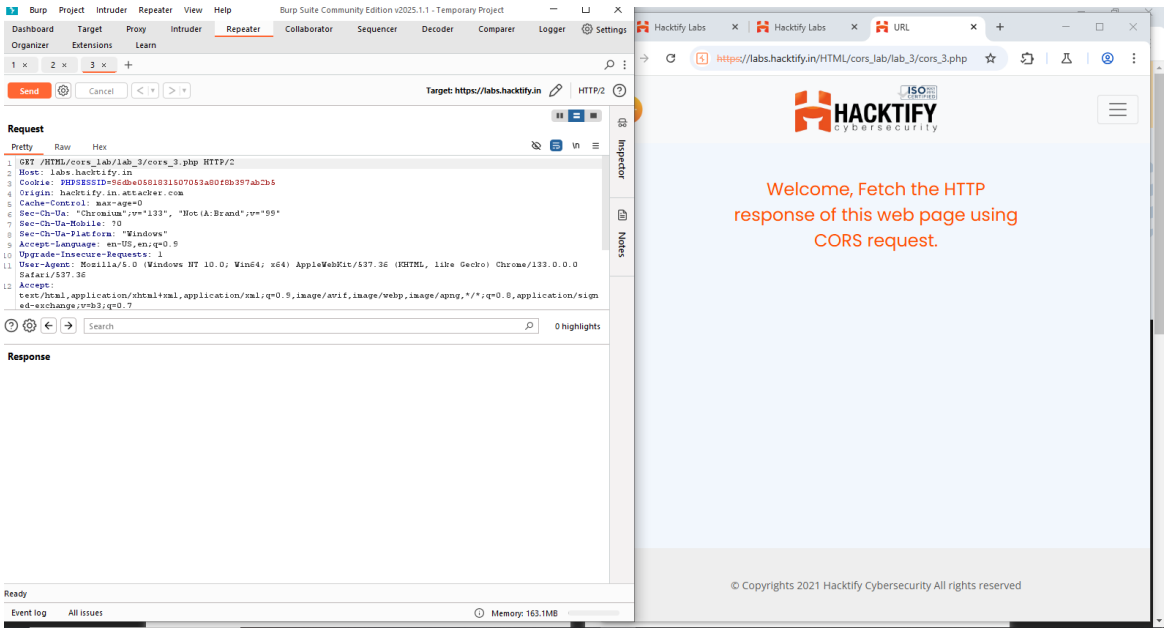


2.3. Cors With prefix Match

Reference	Risk Rating
Cors with prefix match	**medium**
Tools Used	
Manual and Burp-Suite	
Vulnerability Description	
Cross-origin resource sharing (CORS) is a browser mechanism which enables controlled access to resources located outside of a given domain. It extends and adds flexibility to the same-origin policy (SOP). However, it also provides potential for cross-domain attacks, if a website's CORS policy is poorly configured and implemented. CORS is not a protection against cross-origin attacks such as cross-site request forgery (CSRF).	

How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/cors_lab/lab_3/index.php
Consequences of not Fixing the Issue
Cross-origin resource sharing (CORS) is a browser mechanism which enables controlled access to resources located outside of a given domain. It extends and adds flexibility to the same-origin policy (SOP). However, it also provides potential for cross-domain attacks, if a website's CORS policy is poorly configured and implemented. CORS is not a protection against cross-origin attacks such as cross-site request forgery (CSRF).
Suggested Countermeasures
Implement proper CORS headers: The server can add appropriate CORS headers to allow cross-origin requests from only trusted sites. Restrict access to sensitive data: It is important to restrict access to sensitive data to only trusted domains. This can be done by implementing access control measures such as authentication and authorization.
References
https://www.freecodecamp.org/news/exploiting-cors-guide-to-pentesting/

Proof of Concept

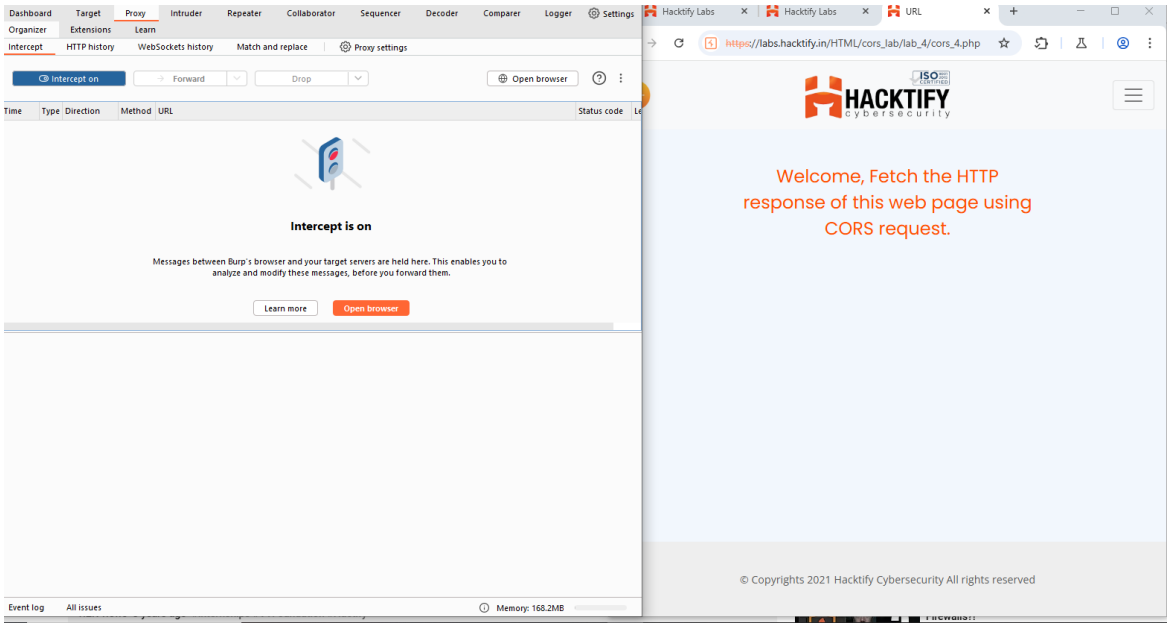


2.4. CORS with suffix match

Reference	Risk Rating
CORS with suffix match	**Medium**
Tools Used	
Manual and Burp-Suite	
Vulnerability Description	
CORS is a security feature created to selectively relax the SOP restrictions and enable controlled access to resources from different domains. CORS rules allow domains to specify which domains can request information from them by adding specific HTTP headers in the response.	

How It Was Discovered
Manual Analysis
Vulnerable URLs
https://www.freecodecamp.org/news/exploiting-cors-guide-to-pentesting/
Consequences of not Fixing the Issue
Remote code execution, and data theft.
Suggested Countermeasures
Specify the allowed origins, only allow trusted sites and Don't whitelist "null".
References
https://www.comparitech.com/blog/information-security/cors-attacks-prevent/

Proof of Concept

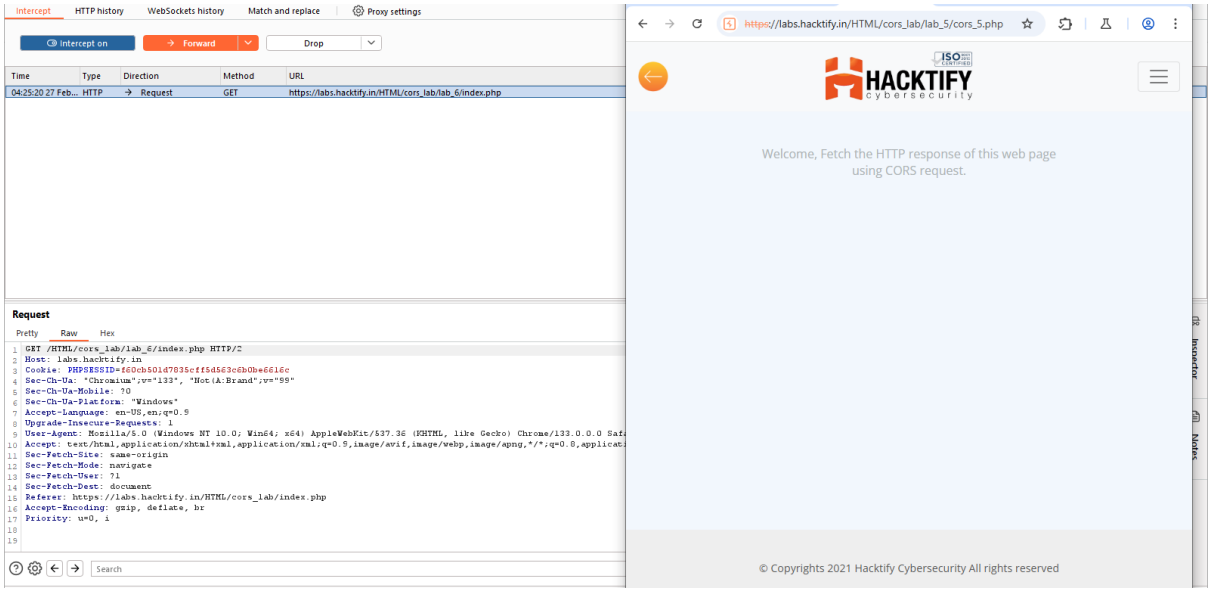


2.5. CORS with Escape dot

Reference	Risk Rating
CORS with Escape dot	**Hard**
Tools Used	
Manual and Burp-Suite	
Vulnerability Description	
This vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin. This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_5/index.php	
Consequences of not Fixing the Issue	

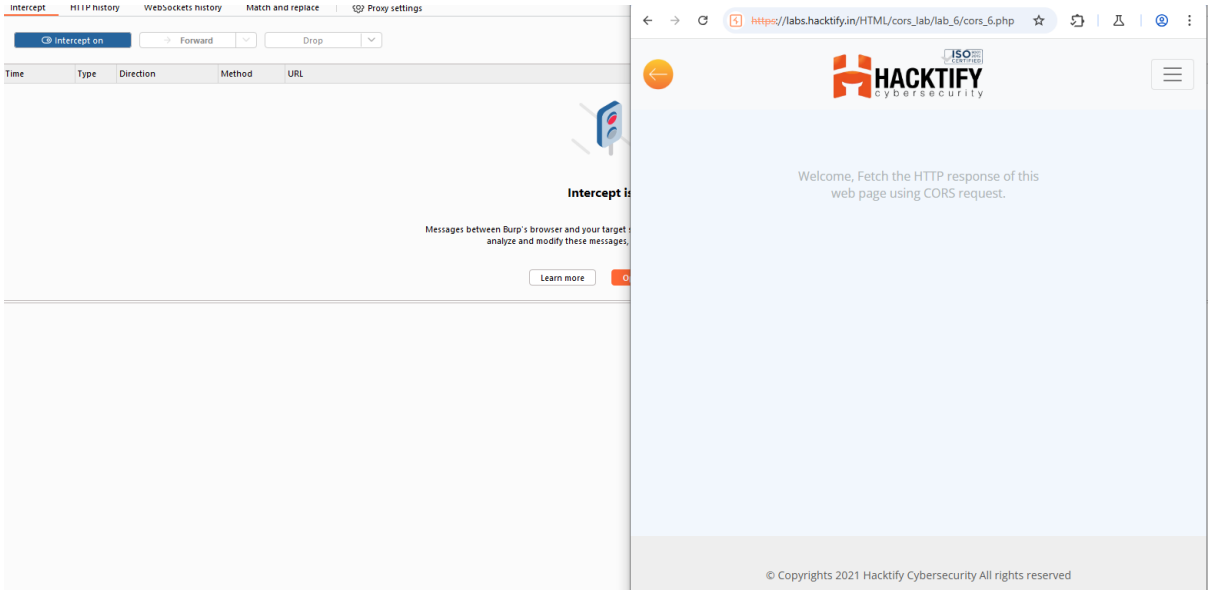
A CORS misconfiguration can leave the application at a high risk of compromises resulting in an impact on the confidentiality and integrity of data by allowing third-party sites to carry out privileged requests through your website's authenticated users such as retrieving user setting information or saved payment card.
Suggested Countermeasures
To mitigate the risk of CORS, it is recommended to whitelist your Access-Control-Allow-Origin instead of wildcarding. Using a subdomain such as subdomain.yoursite.com makes it more difficult for the attackers given they would need to find a vulnerability (such as cross-site scripting or cross-site request forgery) to issue the cross-origin request
References
https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors/

Proof of Concept



2.7. CORS with Arbitrary Subdomain

Reference	Risk Rating
CORS with Arbitrary Subdomain	**Hard**
Tools Used	
Manual and Burp-Suite	
Vulnerability Description	
CORS issues happen when a web server is set up to share resources with other websites in a way that's too open or not specific enough. This means that attackers might trick browsers into sending requests from an authenticated session to the wrong place, which could let them perform actions the user didn't intend.	

How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/cors_lab/lab_7/index.php
Consequences of not Fixing the Issue
A CORS misconfiguration can leave the application at a high risk of compromises resulting in an impact on the confidentiality and integrity of data by allowing third-party sites to carry out privileged requests through your website's authenticated users such as retrieving user setting information or saved payment card.
Suggested Countermeasures
Ensure that you set specific domains for Access-Control-Allow-Origin, by thoroughly testing for CORS vulnerabilities using tools like Burp Suite and applying targeted fixes for each web application stack, you can significantly reduce the risk of cross-origin attacks.
References
https://docs.escape.tech/documentation/dast/vulnerabilities/protocol/cors/

Proof of Concept